

SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE:** CSA13

COURSE OUTCOME COVERED

CO1: *Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Simulation

Code of Conduct:

I _Pusalapati Chandu_ (192372119) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Pchandu*

- 1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs. An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed*, *Opening*, *Open*, and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.
- 2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.
- 3 Design DFA using simulator to accept the string the end with ab over set $\{a,b\}$ $W=$ aaabab
- 4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings "c", "bc", and "bcaaa" over the alphabet $\{a, b, c\}$. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string "c" should be accepted directly from the start state, while "bc" should pass through states representing "b" and then "c". Similarly, "bcaaa" should be processed through a sequence of states that track each additional "a" after "bc". Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.
- 5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet $\{a, b\}$ that begin with either "a" or "b". The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either "a" or "b". After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of "a" and "b", thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

Question 1 — NFA for an Automatic Door System (5 Marks)

Design a Non-Deterministic Finite Automaton (NFA) for an automatic door system that reacts to a sensor detecting a person (P) or no person (N). The system has four states: Closed, Opening, Open, and Closing.

1.1 States and Alphabet

- States (Q) = {Closed, Opening, Open, Closing}
- Input alphabet (Σ) = {P, N} — P = person detected, N = no person detected
- Start state (q_0) = Closed
- Accepting / goal state (F) = {Open} — the state representing the door's fully operational (open) condition

1.2 Formal Definition

NFA $M = (Q, \Sigma, \delta, q_0, F)$

where Q , Σ , q_0 and F are as listed above, and $\delta : Q \times \Sigma \rightarrow 2^Q$ is the (possibly multi-valued) transition function defined below.

1.3 Transition Table

Current State	Input = P	Input = N
Closed	Opening	Closed
Opening	Open	Open
Open	{Open, Closing}	{Open, Closing}
Closing	Closing	Closed

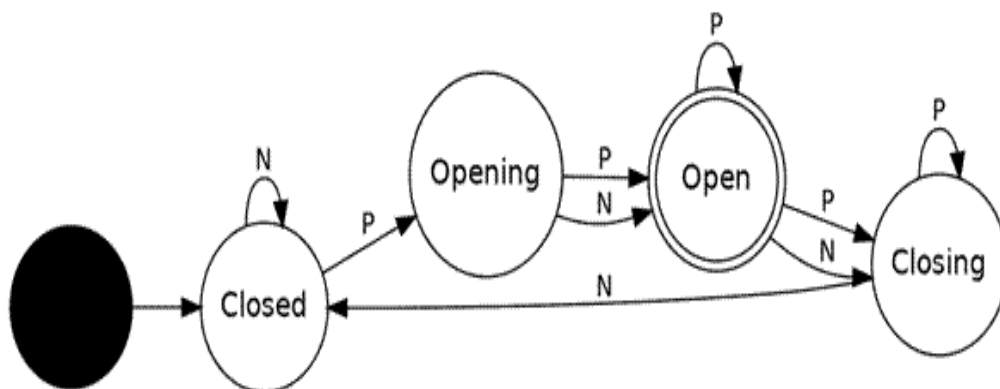


Fig. 1 — State diagram of the Automatic Door NFA

1.4 Explanation of Non-Determinism

- From Closed, sensing P moves the system toward Opening; sensing N keeps it Closed.
- From Opening, the system always progresses to Open (the door finishes opening).
- The genuine non-determinism is modeled at the Open state: on either P or N, the automaton may choose to stay in Open OR move to Closing. This captures real-world ambiguity — e.g. a short sensor glitch, a person lingering near the door, or a timeout beginning to elapse — where more than one next state is possible for the same input.

- From Closing, receiving N (no person) lets the door finish closing and return to Closed; receiving P keeps it in Closing (someone is still near the door).

Because at least one state (Open) has more than one possible next state for the same input symbol, this automaton is a true NFA rather than a DFA.

Question 2 — Finite Automaton for a Traffic Light Control System (5 Marks)

Design a finite automaton to model a traffic light at a road intersection, cycling through Red → Green → Yellow → Red, driven by a timer/control signal.

2.1 States and Alphabet

- States (Q) = {Red, Green, Yellow}
- Input alphabet (Σ) = {t}, where t = 'timer expires' / control-signal event
- Start state (q_0) = Red
- Since this is a cyclic controller (not a language acceptor) every state is a valid/'live' state; there is no explicit reject state.

2.2 Transition Table

Current State	Input = t (timer expires)	Duration Assigned
Red	Green	30 seconds
Green	Yellow	25 seconds
Yellow	Red	5 seconds

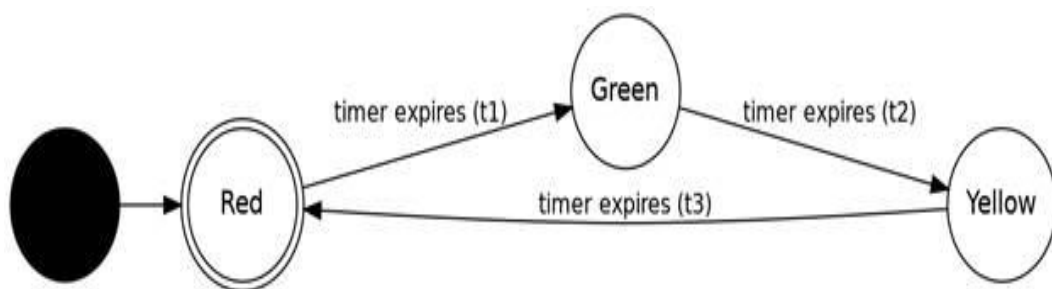


Fig. 2 — State diagram of the Traffic Light FSM

2.3 Timing Behaviour and Continuous Cycling

- Each state is associated with a fixed dwell time (e.g. Red = 30 s, Green = 25 s, Yellow = 5 s). An internal timer counts down while the FSM is in that state.
- When the timer reaches zero, the single input symbol 't' fires and the automaton deterministically moves to the next state in the cycle Red → Green → Yellow → Red, repeating indefinitely — this is why the machine has no final/accepting state; it runs forever as a controller.

2.4 Extending the Model

- Pedestrian crossing: add a 'Pedestrian-Walk' state reachable from Red (or inserted between Red and Green) triggered by a pedestrian push-button input (B). While active, all vehicle directions stay Red and a 'Walk' signal is shown; it transitions back to Red after a fixed walk timer.
- Emergency vehicle override: add a global input (E) accepted from any state, transitioning immediately to a dedicated 'Emergency-Red' state that forces the intersection to all-Red. Once the emergency clears (input C), the FSM resumes the cycle from Red, guaranteeing the interrupted sequence restarts safely rather than resuming mid-cycle.

Question 3 — DFA to Accept Strings Ending in "ab" over $\Sigma = \{a, b\}$ (5 Marks)

Design (and simulate) a DFA that accepts exactly the strings ending with 'ab', and trace the string $W = aaabab$ through it.

3.1 Formal Definition

- $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, start state = q_0 , $F = \{q_2\}$

Intuition: q_0 = 'last symbol seen (if any) is not part of ab-suffix', q_1 = 'last symbol was a', q_2 = 'last two symbols were ab' (accepting).

3.2 Transition Table

State	a	b
q_0 (start)	q_1	q_0
q_1	q_1	q_2 (accept)
q_2 (accept)	q_1	q_0

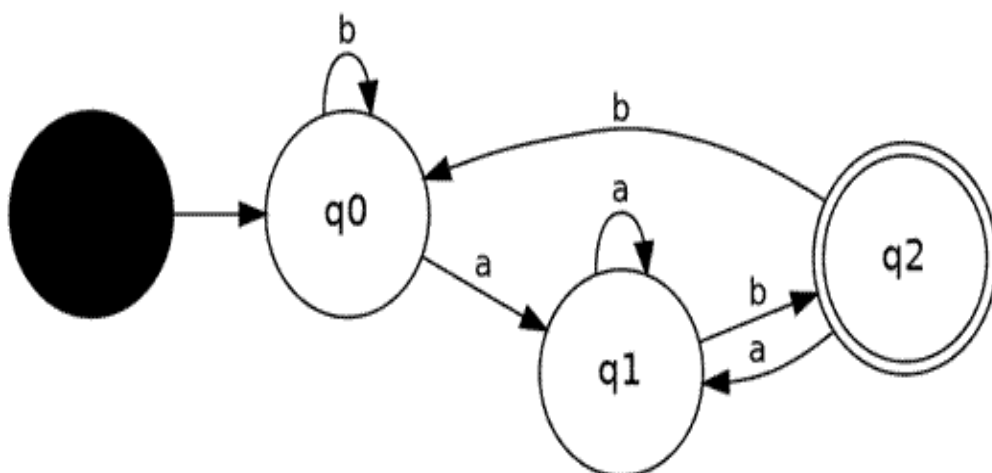


Fig. 3 — DFA accepting strings ending in "ab"

3.3 Simulator Trace for $W = \text{aaabab}$

Step	Current State	Input Symbol	Next State
1	q0	a	q1
2	q1	a	q1
3	q1	a	q1
4	q1	b	q2
5	q2	a	q1
6	q1	b	q2

Final state reached = q2, which is an accepting state. Result: the string "aaabab" IS ACCEPTED, because it ends with the substring "ab" — consistent with running this trace on a simulator such as Autosim/JFLAP.

Question 4 — DFA to Accept Only {"c", "bc", "bcaaa"} over $\Sigma = \{a, b, c\}$ (5 Marks)

Design a DFA using a simulator that accepts only the exact strings 'c', 'bc' and 'bcaaa', rejecting every other string (including all prefixes/extensions of these words).

4.1 Design Strategy

Because only three exact strings must be accepted, the DFA is built as a 'prefix chain' (like a trie) that tracks progress through each target string, together with a single dead/trap state that absorbs any input that deviates from the three valid paths.

- $Q = \{q_0, q_C, q_B, q_{BC}, q_{A1}, q_{A2}, q_{A3}, \text{Dead}\}$
- $\Sigma = \{a, b, c\}$
- Start state = q0
- $F = \{q_C, q_{BC}, q_{A3}\}$ (accepting exactly after "c", "bc", and "bcaaa" respectively)

4.2 Transition Table

State	a	b	c
q0 (start)	Dead	qB	qC (accept: "c")
qC (accept)	Dead	Dead	Dead
qB	Dead	Dead	qBC (accept: "bc")
qBC (accept)	qA1	Dead	Dead

qA1	qA2	Dead	Dead
qA2	qA3 (accept: "bcaaa")	Dead	Dead
qA3 (accept)	Dead	Dead	Dead
Dead	Dead	Dead	Dead

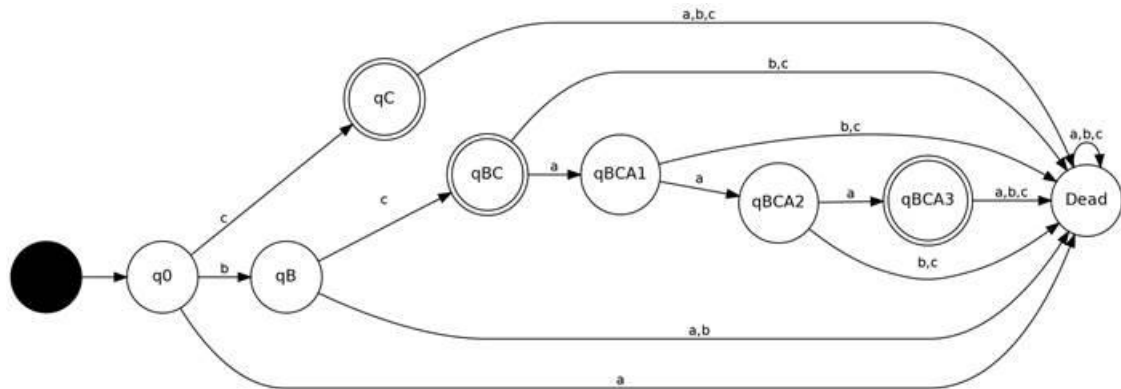


Fig. 4 — DFA accepting exactly $\{c, bc, bcaaa\}$

4.3 Why Each String Is Handled Correctly

- "c": $q_0 \xrightarrow{c} q_C$. q_C is accepting and no further input is expected, so the string "c" alone is accepted; any extra symbol after it sends the machine to Dead.
- "bc": $q_0 \xrightarrow{b} q_B \xrightarrow{c} q_{BC}$. q_{BC} is accepting, matching the required exact string "bc".
- "bcaaa": $q_0 \xrightarrow{b} q_B \xrightarrow{c} q_{BC} \xrightarrow{a} q_{BCA1} \xrightarrow{a} q_{BCA2} \xrightarrow{a} q_{BCA3}$ (accepting), tracking each of the three required a's after "bc".
- Any deviation — wrong symbol at any step, or extra symbols after an accepting state — routes to Dead, which loops on itself and never accepts, guaranteeing no other string is accepted.

Question 5 — DFA to Accept All Strings over $\{a, b\}$ Starting with "a" or "b" (5 Marks)

Design a DFA using a simulator that accepts all strings over $\Sigma = \{a, b\}$ whose first symbol is 'a' or 'b'.

5.1 Key Observation

Since the alphabet itself is exactly $\{a, b\}$, every non-empty string necessarily begins with either 'a' or 'b'. Hence this language is simply "all non-empty strings over $\{a, b\}$ " — the only string excluded is the empty string ϵ .

5.2 Formal Definition

- $Q = \{q_0, q_1\}$, $\Sigma = \{a, b\}$, start state = q_0 , $F = \{q_1\}$

q_0 represents 'no symbol read yet' (non-accepting, so ϵ is rejected). Once the first symbol (a or b) is read, the DFA moves to q_1 and stays there permanently, accepting any continuation.

5.3 Transition Table

State	a	b
q0 (start)	q1 (accept)	q1 (accept)
q1 (accept)	q1 (accept)	q1 (accept)

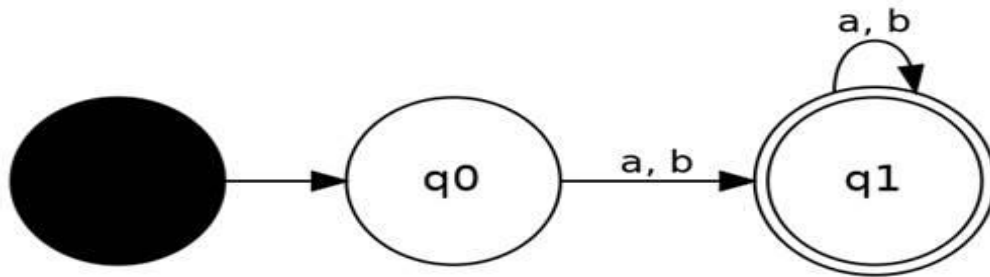


Fig. 5 — DFA accepting all strings starting with a or b

5.4 Verification

- Empty string ϵ : machine stays in q0 (non-accepting) $\rightarrow$ correctly REJECTED.
- Any string such as "a", "b", "ab", "bba...": the very first symbol moves the DFA to accepting state q1, and it remains accepting for every subsequent symbol $\rightarrow$ correctly ACCEPTED.
- An optional explicit Dead state can be added for any symbol outside $\{a, b\}$ if the simulator requires total transition functions, but since Σ contains only a and b, it is not strictly necessary here.